torch.linalg.householder_product#

https://pytorch.org/docs/stable/generated/torch.linalg.householder_product.h

Description:

Computes the first n columns of a product of Householder matrices. Let K\mathbb{K}K be R\mathbb{R}R or C\mathbb{C}C, and let A∈Km×nA \in \mathbb{K}^{m \times n}A∈Km×n be a matrix with columns ai∈Kma_i \in \mathbb{K}^mai∈Km for i=1,…,mi=1,\ldots,mi=1,…,m with m≥nm \geq nm≥n. Denote by bib_ibi the vector resulting from zeroing out the first i−1i-1i−1 components of aia_iai and setting to 1 the iii-th. For a vector τ∈Kk\tau \in \mathbb{K}^kτ∈Kk with k≤nk \leq nk≤n, this function computes the first nnn columns of the matrix where Im\mathrm{I}_mIm is the m-dimensional identity matrix and bHb^{\text{H}}bH is the conjugate transpose when bbb is complex, and the transpose when bbb is real-valued. The output matrix is the same size as the input matrix A. See Representation of Orthogonal or Unitary Matrices for further details. Supports inputs of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if the inputs are batches of matrices then the output has the same batch dimensions.

Function Signature

```
torch.linalg.householder_product(A, tau, *, out=None) → Tensor#

Parameters

Keyword Arguments

Raises
```

Parameters

Keyword

out (Tensor, optional) – output tensor. Ignored if None. Default: None.

Code Examples

```
>>> A = torch.randn(2, 2)
>>> h, tau = torch.geqrf(A)
>>> Q = torch.linalg.householder_product(h, tau)
>>> torch.dist(Q, torch.linalg.qr(A).Q)
tensor(0.)

>>> h = torch.randn(3, 2, 2, dtype=torch.complex128)
>>> tau = torch.randn(3, 1, dtype=torch.complex128)
>>> Q = torch.linalg.householder_product(h, tau)
>>> Q
tensor([[[ 1.8034+0.4184j,  0.2588-1.0174j],
         [-0.6853+0.7953j,  2.0790+0.5620j]],

         [[ 1.4581+1.6989j, -1.5360+0.1193j],
         [ 1.3877-0.6691j,  1.3512+1.3024j]],

         [[ 1.4766+0.5783j,  0.0361+0.6587j],
         [ 0.6396+0.1612j,  1.3693+0.4481j]]],
dtype=torch.complex128)
```

Notes & Warnings

NOTE

See also torch.geqrf() can be used together with this function to form the Q from the qr() decomposition. torch.ormqr() is a related function that computes the matrix multiplication of a product of Householder matrices with another matrix. However, that function is not supported by autograd.

⚠ WARNING

Warning Gradient computations are only well-defined if τi≠1 ∣ ∣ ai ∣ ∣ 2\tau_i \neq \frac{1}{||a_i||^2}τi= ∣ ∣ ai ∣ ∣ 21. If this condition is not met, no error will be thrown, but the gradient produced may contain NaN.

Detailed Documentation

Documentation

Computes the first n columns of a product of Householder matrices.

Let K\mathbb{K}K be R\mathbb{R}R or C\mathbb{C}C, and let A∈Km×nA \in \mathbb{K}^{m \times n}A∈Km×n be a matrix with columns ai∈Kma_i \in \mathbb{K}^mai∈Km for i=1,…,mi=1,\ldots,mi=1,…,m with m≥nm \geq nm≥n. Denote by bib_ibi the vector resulting from zeroing out the first i−1i-1i−1 components of aia_iai and setting to 1 the iii-th. For a vector τ∈Kk\tau \in \mathbb{K}^kτ∈Kk with k≤nk \leq nk≤n, this function computes the first nnn columns of the matrix

where Im\mathrm{I}_mIm is the m-dimensional identity matrix and bHb^{\text{H}}bH is the conjugate transpose when bbb is complex, and the transpose when bbb is real-valued. The output matrix is the same size as the input matrix A.

See Representation of Orthogonal or Unitary Matrices for further details.

Supports inputs of float, double, cfloat and cdouble dtypes. Also supports batches of matrices, and if the inputs are batches of matrices then the output has the same batch dimensions.

torch.geqrf() can be used together with this function to form the Q from the qr() decomposition.

torch.ormqr() is a related function that computes the matrix multiplication of a product of Householder matrices with another matrix. However, that function is not supported by autograd.

Gradient computations are only well-defined if τi≠1 ∣ ∣ ai ∣ ∣ 2\tau_i \neq \frac{1}{||a_i||^2}τi= ∣ ∣ ai ∣ ∣ 21. If this condition is not met, no error will be thrown, but the gradient produced may contain NaN.

A (Tensor) – tensor of shape (*, m, n) where * is zero or more batch dimensions.

tau (Tensor) – tensor of shape (*, k) where * is zero or more batch dimensions.

out (Tensor, optional) – output tensor. Ignored if None. Default: None.

RuntimeError – if A doesn’t satisfy the requirement m >= n, or tau doesn’t satisfy the requirement n >= k.